

SOLUCIÓN DEL EXAMEN — LARAVEL

Plataforma de Gestión de Música

Documento de uso exclusivo del profesorado. No distribuir al alumnado.

PARTE 1 — Tutorial de despliegue (4 puntos)

Paso 1 — Migración `create_albums_table`

Fichero: `database/migrations/xxxx_xx_xx_create_albums_table.php`

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('albums', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->string('title');
            $table->string('artist');
            $table->string('genre');
            $table->smallInteger('release_year');
            $table->string('record_label')->nullable();
            $table->text('description')->nullable();
            $table->string('cover')->nullable();
            $table->decimal('average_rating', 3, 2)->default(0);
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('albums');
    }
};
```

Paso 2 — Modelo `Album`

Fichero: `app/Models/Album.php`

```
<?php
```

```

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Album extends Model
{
    use HasFactory;

    protected $fillable = [
        'user_id',
        'title',
        'artist',
        'genre',
        'release_year',
        'record_label',
        'description',
        'cover',
        'average_rating',
    ];

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}

```

Paso 3 — Seeder AlbumSeeder

Fichero: database/seeder/AlbumSeeder.php

```

<?php

namespace Database\Seeders;

use App\Models\Album;
use App\Models\User;
use Illuminate\Database\Seeder;

class AlbumSeeder extends Seeder
{
    public function run(): void
    {
        $user = User::first() ?? User::factory()->create([
            'name'      => 'Usuario Demo',
            'email'     => 'demo@musica.com',
            'password' => bcrypt('password'),
        ]);

        $albums = [

```

```

        ['title' => 'Thriller',          'artist' => 'Michael Jackson',
'genre' => 'Pop',          'release_year' => 1982, 'average_rating' => 4.8],
        ['title' => 'Dark Side of the Moon', 'artist' => 'Pink Floyd',
'genre' => 'Rock',          'release_year' => 1973, 'average_rating' => 4.9],
        ['title' => 'Kind of Blue',          'artist' => 'Miles Davis',
'genre' => 'Jazz',          'release_year' => 1959, 'average_rating' => 4.7],
        ['title' => 'Nevermind',          'artist' => 'Nirvana',
'genre' => 'Rock',          'release_year' => 1991, 'average_rating' => 4.6],
        ['title' => 'Random Access Memories', 'artist' => 'Daft Punk',
'genre' => 'Electronica', 'release_year' => 2013, 'average_rating' => 4.5],
        ['title' => 'Back in Black',          'artist' => 'AC/DC',
'genre' => 'Rock',          'release_year' => 1980, 'average_rating' => 4.7],
        ['title' => 'Rumours',          'artist' => 'Fleetwood Mac',
'genre' => 'Pop',          'release_year' => 1977, 'average_rating' => 4.6],
        ['title' => 'Abbey Road',          'artist' => 'The Beatles',
'genre' => 'Pop',          'release_year' => 1969, 'average_rating' => 4.9],
        ['title' => 'To Pimp a Butterfly', 'artist' => 'Kendrick Lamar',
'genre' => 'Hip-Hop', 'release_year' => 2015, 'average_rating' => 4.8],
        ['title' => 'Homogenic',          'artist' => 'Bjork',
'genre' => 'Electronica', 'release_year' => 1997, 'average_rating' => 4.4],
    ];

    foreach ($albums as $data) {
        Album::create(array_merge($data, ['user_id' => $user->id]));
    }
}
}

```

DatabaseSeeder.php — añadir en `run()` :

```
$this->call(AlbumSeeder::class);
```

Paso 4 — Migración `add_role_to_users_table`

Fichero: `database/migrations/xxxx_xx_xx_add_role_to_users_table.php`

```

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('role')->default('user')->after('email');
        });
    }
}

```

```
public function down(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->dropColumn('role');
    });
}
};
```

Paso 5 — Métodos en el modelo `User`

Fichero: `app/Models/User.php` — añadir dentro de la clase:

```
// Añadir al bloque de imports:
use Illuminate\Database\Eloquent\Relations\HasMany;

public function isAdmin(): bool
{
    return $this->role === 'admin';
}

public function albums(): HasMany
{
    return $this->hasMany(Album::class);
}
```

Paso 6 — Seeder `AdminSeeder`

Fichero: `database/seeder/AdminSeeder.php`

```
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class AdminSeeder extends Seeder
{
    public function run(): void
    {
        User::create([
            'name'      => 'Administrador',
            'email'     => 'admin@musica.com',
            'password' => Hash::make('password'),
            'role'      => 'admin',
        ]);
    }
}
```

```
}  
}
```

DatabaseSeeder.php — añadir en `run()` :

```
$this->call(AdminSeeder::class);
```

Paso 7 — Middleware **IsAdmin**

Fichero: `app/Http/Middleware/IsAdmin.php`

```
<?php  
  
namespace App\Http\Middleware;  
  
use Closure;  
use Illuminate\Http\Request;  
use Symfony\Component\HttpFoundation\Response;  
  
class IsAdmin  
{  
    public function handle(Request $request, Closure $next): Response  
    {  
        if (!auth()->check() || !auth()->user()->isAdmin()) {  
            abort(403, 'No tienes permisos para acceder a esta seccion.');        }  
  
        return $next($request);  
    }  
}
```

bootstrap/app.php — dentro de `withMiddleware()` :

```
->withMiddleware(function (Middleware $middleware) {  
    $middleware->alias([  
        'admin' => \App\Http\Middleware\IsAdmin::class,  
    ]);  
})
```

Paso 8 — Form Request **StoreAlbumRequest**

Fichero: `app/Http/Requests/StoreAlbumRequest.php`

```
<?php  
  
namespace App\Http\Requests;  
  
use Illuminate\Foundation\Http\FormRequest;
```

```

class StoreAlbumRequest extends FormRequest
{
    public function authorize(): bool
    {
        return auth()->check();
    }

    public function rules(): array
    {
        return [
            'title'          => ['required', 'string', 'max:200'],
            'artist'         => ['required', 'string', 'max:200'],
            'genre'          => ['required', 'string', 'max:100'],
            'release_year' => ['required', 'integer', 'min:1900', 'max:' .
date('Y')],
            'record_label' => ['nullable', 'string', 'max:150'],
            'description' => ['nullable', 'string', 'max:2000'],
            'cover'         => ['nullable', 'image', 'mimes:jpg,jpeg,png,webp',
'max:2048'],
        ];
    }

    public function messages(): array
    {
        return [
            'title.required'      => 'El titulo del album es obligatorio.',
            'artist.required'     => 'El nombre del artista es obligatorio.',
            'genre.required'      => 'El genero musical es obligatorio.',
            'release_year.required' => 'El anio de lanzamiento es obligatorio.',
            'release_year.min'    => 'El anio no puede ser anterior a 1900.',
            'cover.image'         => 'La portada debe ser una imagen valida.',
            'cover.max'           => 'La portada no puede superar los 2 MB.',
        ];
    }
}

```

Paso 9 — Controlador AlbumController

Fichero: app/Http/Controllers/AlbumController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\Album;
use App\Http\Requests\StoreAlbumRequest;
use Illuminate\Http\Request;
use Illuminate\View\View;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Storage;

```

```

class AlbumController extends Controller
{
    public function index(): View
    {
        $albums = Album::orderByDesc('created_at')->paginate(10);
        return view('albums.index', compact('albums'));
    }

    public function create(): View
    {
        return view('albums.create');
    }

    public function store(StoreAlbumRequest $request): RedirectResponse
    {
        $data = $request->validated();
        $data['user_id'] = auth()->id();

        if ($request->hasFile('cover')) {
            $data['cover'] = $request->file('cover')->store('covers', 'public');
        }

        Album::create($data);

        return redirect()->route('albums.index')
            ->with('success', 'Album creado correctamente.');
```

```

    }

    public function show(Album $album): View
    {
        return view('albums.show', compact('album'));
    }

    public function edit(Album $album): View
    {
        $this->authorizeAccess($album);
        return view('albums.edit', compact('album'));
    }

    public function update(StoreAlbumRequest $request, Album $album):
RedirectResponse
    {
        $this->authorizeAccess($album);

        $data = $request->validated();

        if ($request->hasFile('cover')) {
            if ($album->cover) {
                Storage::disk('public')->delete($album->cover);
            }
            $data['cover'] = $request->file('cover')->store('covers', 'public');
        }
    }
}

```

```

        $album->update($data);

        return redirect()->route('albums.show', $album)
            ->with('success', 'Album actualizado correctamente.');
```

```

    }

```

```

public function destroy(Album $album): RedirectResponse
{

```

```

    $this->authorizeAccess($album);

```

```

    if ($album->cover) {
        Storage::disk('public')->delete($album->cover);
    }

```

```

    $album->delete();

```

```

    return redirect()->route('albums.index')
        ->with('success', 'Album eliminado.');
```

```

}

```

```

public function search(Request $request): View
{

```

```

    $query = Album::query();

```

```

    $query->when($request->filled('q'), function ($q) use ($request) {
        $q->where(function ($sub) use ($request) {
            $sub->where('title', 'like', '%' . $request->q . '%')
                ->orWhere('artist', 'like', '%' . $request->q . '%');
        });
    });

```

```

    $query->when($request->filled('genre'), function ($q) use ($request) {
        $q->where('genre', $request->genre);
    });

```

```

    $query->when($request->filled('year_from'), function ($q) use ($request) {
        $q->where('release_year', '>=', $request->year_from);
    });

```

```

    $query->when($request->filled('year_to'), function ($q) use ($request) {
        $q->where('release_year', '<=', $request->year_to);
    });

```

```

    $query->when($request->filled('min_rating'), function ($q) use ($request) {
        $q->where('average_rating', '>=', $request->min_rating);
    });

```

```

    $query->when($request->filled('sort'), function ($q) use ($request) {
        match ($request->sort) {
            'title'      => $q->orderBy('title'),
            'year_desc'  => $q->orderByDesc('release_year'),

```

```

        'rating_desc' => $q->orderByDesc('average_rating'),
        default       => $q->orderByDesc('created_at'),
    };
}, function ($q) {
    $q->orderByDesc('created_at');
});

$albums = $query->paginate(10)->withQueryString();
$genres = Album::distinct()->orderBy('genre')->pluck('genre');

return view('albums.search', compact('albums', 'genres'));
}

private function authorizeAccess(Album $album): void
{
    if (auth()->id() !== $album->user_id && !auth()->user()->isAdmin()) {
        abort(403);
    }
}
}
}

```

Paso 10 — Vistas Blade

resources/views/albums/index.blade.php

Igual que el Fragmento J del sobre, con el enlace al buscador añadido en la cabecera:

```

<div class="d-flex gap-2">
    <a href="{{ route('albums.search') }}" class="btn btn-outline-
secondary">Buscar</a>
    @auth
        <a href="{{ route('albums.create') }}" class="btn btn-primary">+ Anadir
album</a>
    @endauth
</div>

```

Las vistas `show.blade.php`, `create.blade.php` y `edit.blade.php` son exactamente los Fragmentos K, L y M del sobre sin modificaciones.

Paso 11 — Rutas en `web.php`

```

use App\Http\Controllers\AlbumController;

// Buscador – ANTES del resource para evitar conflicto con {album}
Route::get('/albums/search', [AlbumController::class, 'search'])->
>name('albums.search');

// Rutas publicas
Route::resource('albums', AlbumController::class)->only(['index', 'show']);

```

```
// Rutas protegidas
Route::middleware('auth')->group(function () {
    Route::resource('albums', AlbumController::class)->except(['index', 'show']);
});
```

PARTE 2 — Resolución de errores (3 puntos)

Error 1 — Método `index()` | `all()->paginate()`

Código con error:

```
$albums = Album::all()->paginate(10);
```

Por que esta mal: `all()` ejecuta inmediatamente la consulta y devuelve una Collection con todos los registros en memoria. Las Collections no tienen el método `paginate()`, que solo existe en el Query Builder. Esto lanza un error fatal en tiempo de ejecución.

Corrección:

```
$albums = Album::paginate(10);
```

Error 2 — Método `update()` | Operador `||` en la autorización

Código con error:

```
if (auth()->id() !== $album->user_id || auth()->user()->isAdmin()) {
    abort(403);
}
```

Por que esta mal: Con `||`, la condición es verdadera (y bloquea el acceso) cuando el usuario ES administrador, que es exactamente lo contrario de lo que se quiere. El administrador debería tener siempre acceso, no ser bloqueado.

Corrección: Cambiar `||` por `&&` y negar `isAdmin()`:

```
if (auth()->id() !== $album->user_id && !auth()->user()->isAdmin()) {
    abort(403);
}
```

Error 3 — Método `show()` | `$reviews` no se pasa a la vista

Código con error:

```
$reviews = $album->reviews()->orderBy('created_at', 'desc')->get();
return view('albums.show', compact('album'));
```

Por que esta mal: La variable `$reviews` se calcula pero no se incluye en el `compact()`, por lo que nunca llega a la vista. Cualquier intento de usarla en Blade lanzara un error de variable no definida.

Correccion:

```
$reviews = $album->reviews()->orderBy('created_at', 'desc')->get();
return view('albums.show', compact('album', 'reviews'));
```

PARTE 3 — Buscador con filtros avanzados (3 puntos)

Ruta en `web.php`

```
// Debe ir ANTES del Route::resource('albums', ...)
Route::get('/albums/search', [AlbumController::class, 'search'])-
>name('albums.search');
```

Método `search()` — ver Paso 9 del controlador arriba

El metodo `search()` ya esta incluido en el controlador completo de la Parte 1. Se acepta cualquier implementacion equivalente que use `when()` para aplicar los filtros de forma condicional y `->paginate(10)->withQueryString()` para la paginacion.

Vista `resources/views/albums/search.blade.php`

```
@extends('layouts.app')

@section('content')
<div class="container py-4">
    <h1 class="mb-4">Buscar albumes</h1>

    <form action="{{ route('albums.search') }}" method="GET" class="card p-4 mb-4 shadow-sm">
        <div class="row g-3">

            <div class="col-md-4">
                <label class="form-label">Texto libre</label>
                <input type="text" name="q" class="form-control"
                    placeholder="Titulo o artista..."
                    value="{{ request('q') }}">
            </div>

            <div class="col-md-2">
                <label class="form-label">Genero</label>
                <select name="genre" class="form-select">
                    <option value="">Todos</option>
                    @foreach($genres as $genre)
                        <option value="{{ $genre }}"
                            {{ request('genre') === $genre ? 'selected' : '' }}>
                            {{ $genre }}
                    @endforeach
                </select>
            </div>
        </div>
    </form>
</div>
```

```

        </option>
    @endforeach
</select>
</div>

<div class="col-md-2">
    <label class="form-label">Año desde</label>
    <input type="number" name="year_from" class="form-control"
        placeholder="1900" min="1900" max="{{ date('Y') }}"
        value="{{ request('year_from') }}">
</div>

<div class="col-md-2">
    <label class="form-label">Año hasta</label>
    <input type="number" name="year_to" class="form-control"
        placeholder="{{ date('Y') }}" min="1900" max="{{ date('Y') }}
    </div>

    <div class="col-md-1">
        <label class="form-label">Val. min.</label>
        <select name="min_rating" class="form-select">
            <option value=""></option>
            @foreach([1,2,3,4,5] as $n)
                <option value="{{ $n }}"
                    {{ request('min_rating') == $n ? 'selected' : '' }}>
                    {{ $n }}
                </option>
            @endforeach
        </select>
    </div>

    <div class="col-md-1">
        <label class="form-label">Ordenar</label>
        <select name="sort" class="form-select">
            <option value="">Recientes</option>
            <option value="title" {{ request('sort') == 'title'
? 'selected' : '' }}>A-Z</option>
            <option value="year_desc" {{ request('sort') == 'year_desc'
? 'selected' : '' }}>Año desc</option>
            <option value="rating_desc" {{ request('sort') == 'rating_desc'
? 'selected' : '' }}>Valoración desc</option>
        </select>
    </div>

</div>

<div class="mt-3 d-flex gap-2">
    <button type="submit" class="btn btn-primary">Buscar</button>
    <a href="{{ route('albums.search') }}" class="btn btn-outline-
secondary">Limpiar filtros</a>

```

```

        <a href="{{ route('albums.index') }}" class="btn btn-outline-secondary
ms-auto">Volver al listado</a>
    </div>
</form>

<p class="text-muted mb-3">
    Se han encontrado <strong>{{ $albums->total() }}</strong>
    {{ $albums->total() === 1 ? 'album' : 'albumes' }}.
</p>

@forelse($albums as $album)
    <div class="card mb-2">
        <div class="card-body d-flex justify-content-between align-items-
center">
            <div>
                <strong>{{ $album->title }}</strong>
                <span class="text-muted ms-2">- {{ $album->artist }}</span>
                <span class="badge bg-secondary ms-2">{{ $album->genre }}</span>
                <span class="text-muted ms-2">{{ $album->release_year }}</span>
            </div>
            <div class="d-flex align-items-center gap-3">
                <span>{{ number_format($album->average_rating, 1) }}
estrellas</span>
                <a href="{{ route('albums.show', $album) }}" class="btn btn-sm
btn-outline-primary">
                    Ver detalle
                </a>
            </div>
        </div>
    </div>
@empty
    <div class="alert alert-info">
        No se han encontrado albumes con los filtros seleccionados.
    </div>
@endforelse

<div class="mt-4">
    {{ $albums->links() }}
</div>
</div>
@endsection

```